To do that, add the following lines of code at the end of the program called '*disgraph9.sagews*'.

```
G.distance('Kolkata', 'Mumbai')
G.distance('Mumbai', 'Kolkata')
G.distance('Mumbai', 'Kavaratti')
```

Upon execution of the modified program, the following additional output will be displayed on the screen.

```
2
2
+Infinity
```

From the output, it is clear that the distance from '*Kolkata*' to '*Mumbai*' and '*Mumbai*' to '*Kolkata*' is the same (2), thus showing the symmetric nature of undirected graphs. Use the *distance( )* method on the undirected graphs we have generated in the previous articles to test that the distance between any pair of vertices is the same in both directions. Furthermore, the distance from '*Mumbai*' to '*Kavaratti*' is defined as infinity because there is no path between '*Mumbai*' and '*Kavaratti*'.

Now, let us test the *distance( )* method on the directed graph *Dcycle_11*. To do that, add the following lines of code at the end of the program called '*dcycle11.sagews*'.

```
DG.distance(1, 3)
DG.distance(3, 1)
```

Upon execution of the modified program, the following additional output will be displayed on the screen.

```
2
6
```

The output shows that the distance from vertex 1 to 3 is 2, while the distance from vertex 3 to 1 is 6, demonstrating the asymmetric nature of directed graphs.

It is worth noting that SageMath, as mentioned earlier, offers special methods for generating various graphs, both undirected and directed. For example, in the first article in this series, we generated a graph known as the icosahedral graph. Similarly, SageMath provides a method called *Circuit( )* to generate directed cycles. Upon execution of the following line of code on CoCalc, we get a directed cycle of length 8 isomorphic to the directed cycle *Dcycle_11*.

```
digraphs.Circuit(8).show( )
```

Let us see one more example. Execute the following line of code on CoCalc to generate a *complete directed graph*. This type of graph has a directed edge from every vertex to every other vertex.

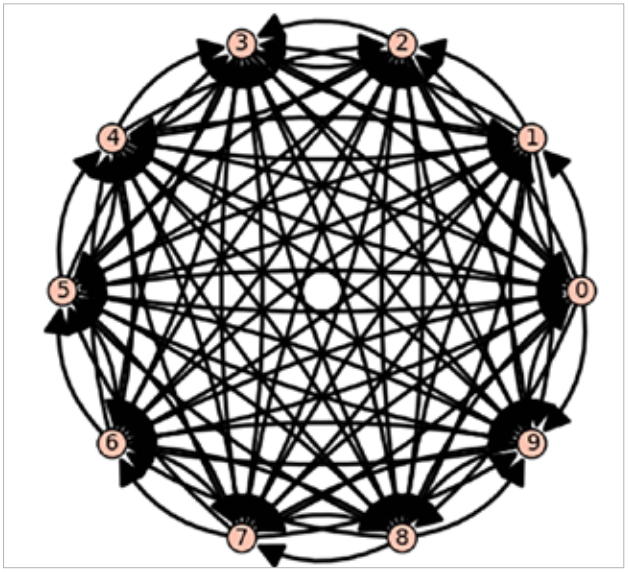

Figure 5: A complete directed graph

```
digraphs.Complete(10).show( )
```

Upon executing the above line of code, we obtain a complete directed graph on 10 vertices, shown in Figure 5. How many edges does this graph have? Please find the answer both mathematically and by executing SageMath code. Additionally, *undirected complete graphs* (often simply called complete graphs) also exist. How many edges are there in an undirected complete graph with 10 vertices? All these questions will be answered in the next article in this series.

In this article, we conducted a thorough investigation of undirected simple graphs as well as multigraphs. We were also introduced to directed graphs, but there is still much more to learn about them. Additionally, we need to discuss mixed graphs and mixed multigraphs, yet another variant. Hence, we will be discussing more about graphs in the next few articles of this series as well.

A small side note: I am finding it increasingly difficult to give meaningful titles to the articles as we delve deeper into exploring graphs extensively. However, let me assure you that it is absolutely essential to do so. As social media explodes, information floods in, and AI takes centre stage, graphs will become crucial tools to understand and make sense of it all. So, in the next article, we will explore many more powerful graph-theoretic methods offered by SageMath. Until then, goodbye. END

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He maintains a technical blog.